

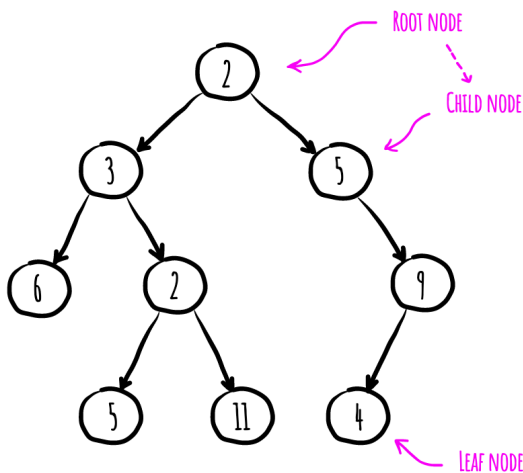
Tree

In computer science, a **tree** is a widely used abstract data type (ADT) — or data structure implementing this ADT—that simulates a hierarchical tree structure, with a root value and subtrees of children with a parent node, represented as a set of linked nodes.

A tree data structure can be defined recursively (locally) as a collection of nodes (starting at a root node), where each node is a data structure consisting of a value, together with a list of references to nodes (the "children"), with the constraints that no reference is duplicated, and none points to the root.

A simple unordered tree; in this diagram, the node labeled 3 has two children, labeled 2 and 6, and one parent, labeled 2. The root node, at the top, has no parent.

TREE



A COLLECTION OF NODES, WHERE EACH NODE IS A DATA STRUCTURE CONSISTING OF A VALUE, TOGETHER WITH A LIST OF REFERENCES TO NODES, WITH THE CONSTRAINTS THAT:

- NO REFERENCE IS DUPLICATED,
- AND NONE POINTS TO THE ROOT.

THERE ARE MANY KINDS OF TREES:

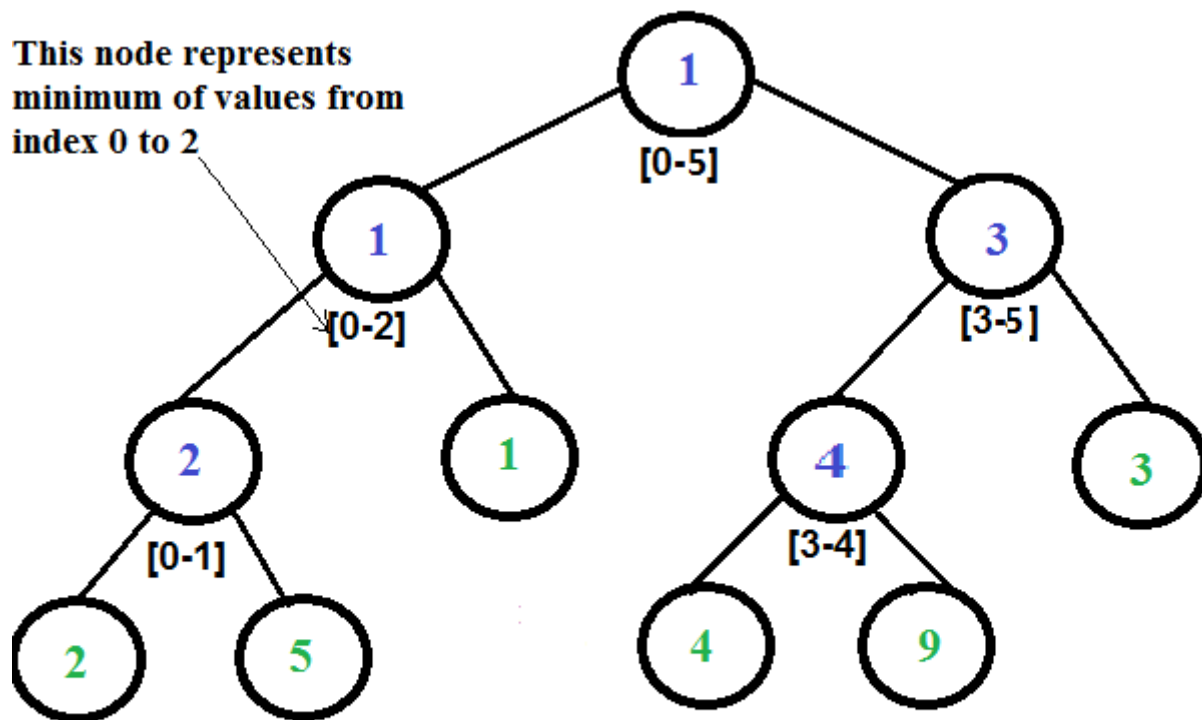


Segment Tree

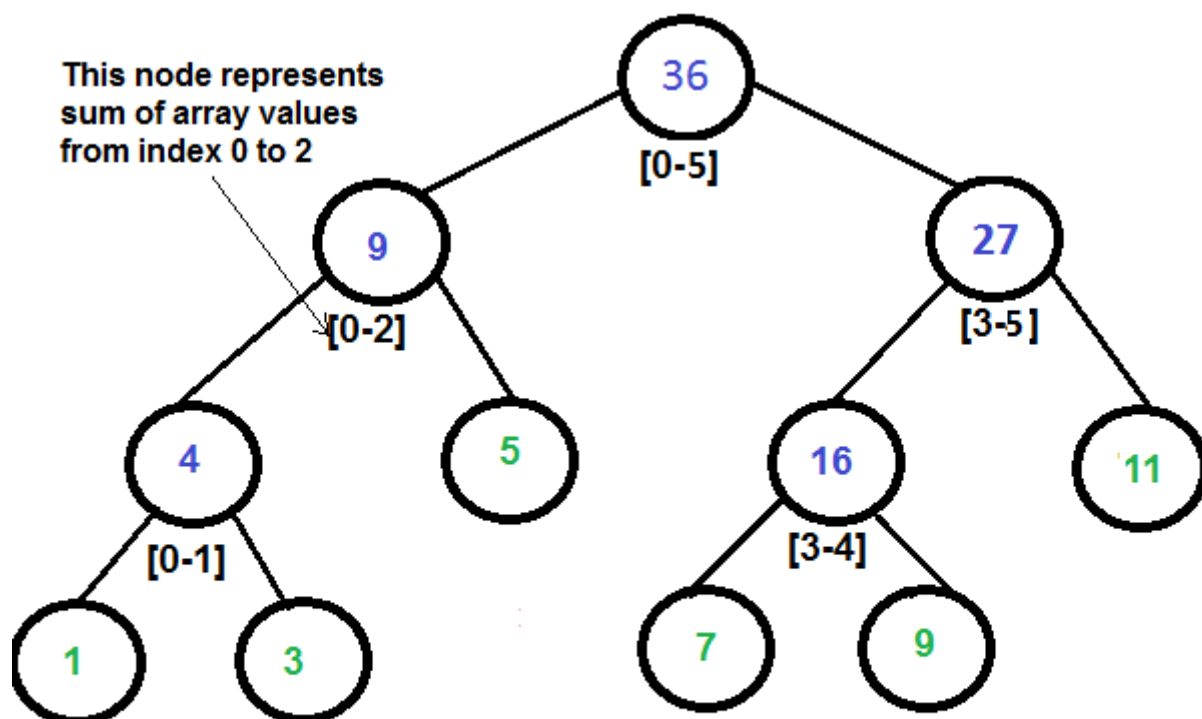
In computer science, a **segment tree** also known as a statistic tree is a tree data structure used for storing information about intervals, or segments. It allows querying which of the stored segments contain a given point. It is, in principle, a static structure; that is, it's a structure that cannot be modified once it's built. A similar data structure is the interval tree.

A segment tree is a binary tree. The root of the tree represents the whole array. The two children of the root represent the first and second halves of the array. Similarly, the children of each node corresponds to the two halves of the array corresponding to the node.

We build the tree bottom up, with the value of each node being the "minimum" (or any other function) of its children's values. This will take $O(n \log n)$ time. The number of operations done is the height of the tree, which is $O(\log n)$. To do range queries, each node splits the query into two parts, one sub-query for each child. If a query contains the whole subarray of a node, we can use the precomputed value at the node. Using this optimization, we can prove that only $O(\log n)$ minimum operations are done.



Segment Tree for input array {2, 5, 1, 4, 9, 3}



Segment Tree for input array {1, 3, 5, 7, 9, 11}

Application

A segment tree is a data structure designed to perform certain array operations efficiently - especially those involving range queries.

Applications of the segment tree are in the areas of computational geometry, and geographic information systems.

Current implementation of Segment Tree implies that you may pass any binary (with two input params) function to it, and thus you're able to do range query for variety of functions. In tests, you may find examples of doing `min`, `max` and `sum` range queries on SegmentTree.

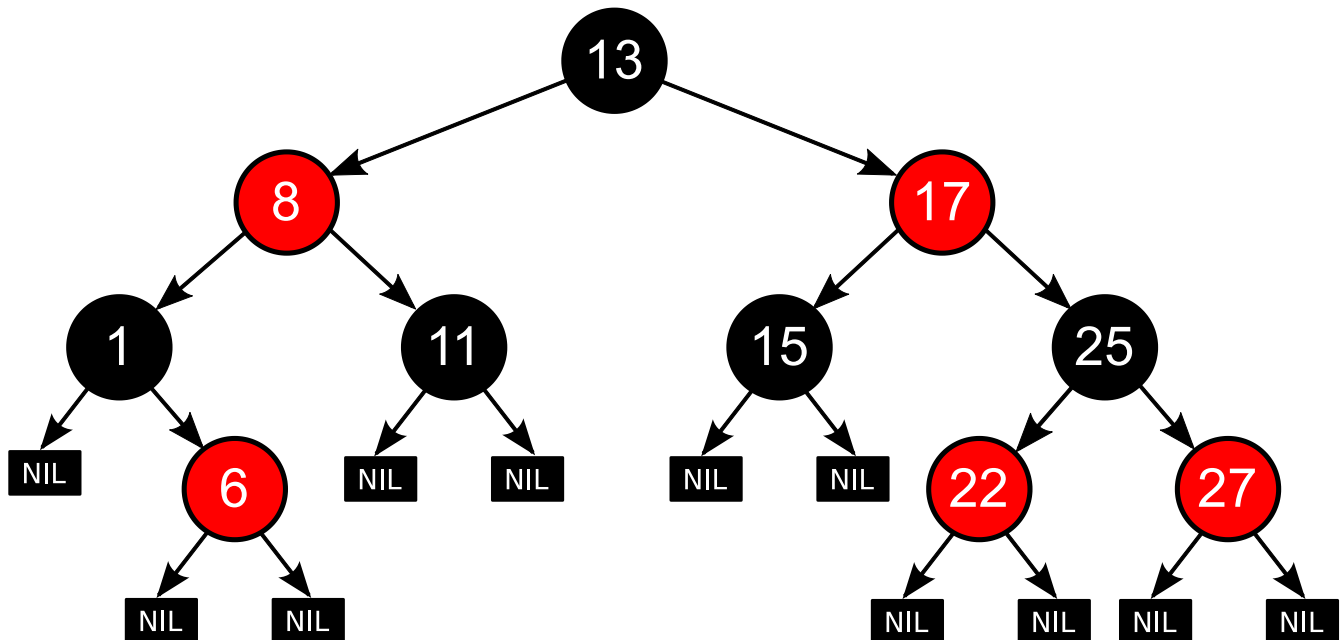
Red–Black Tree

A **red–black tree** is a kind of self-balancing binary search tree in computer science. Each node of the binary tree has an extra bit, and that bit is often interpreted as the color (red or black) of the node. These color bits are used to ensure the tree remains approximately balanced during insertions and deletions.

Balance is preserved by painting each node of the tree with one of two colors in a way that satisfies certain properties, which collectively constrain how unbalanced the tree can become in the worst case. When the tree is modified, the new tree is subsequently rearranged and repainted to restore the coloring properties. The properties are designed in such a way that this rearranging and recoloring can be performed efficiently.

The balancing of the tree is not perfect, but it is good enough to allow it to guarantee searching in $O(\log n)$ time, where n is the total number of elements in the tree. The insertion and deletion operations, along with the tree rearrangement and recoloring, are also performed in $O(\log n)$ time.

An example of a red–black tree:



Properties

In addition to the requirements imposed on a binary search tree the following must be satisfied by a red–black tree:

- Each node is either red or black.
- The root is black. This rule is sometimes omitted. Since the root can always be changed from red to black, but not necessarily vice versa, this rule has little effect on analysis.

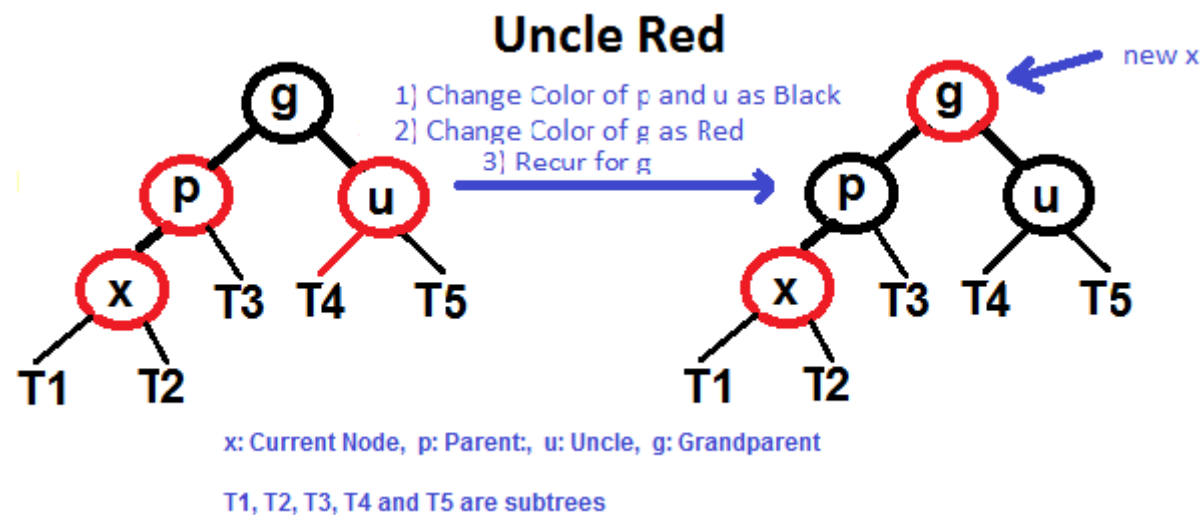
- All leaves (NIL) are black.
- If a node is red, then both its children are black.
- Every path from a given node to any of its descendant NIL nodes contains the same number of black nodes.

Some definitions: the number of black nodes from the root to a node is the node's **black depth**; the uniform number of black nodes in all paths from root to the leaves is called the **black-height** of the red–black tree.

These constraints enforce a critical property of red–black trees: *the path from the root to the farthest leaf is no more than twice as long as the path from the root to the nearest leaf*. The result is that the tree is roughly height-balanced. Since operations such as inserting, deleting, and finding values require worst-case time proportional to the height of the tree, this theoretical upper bound on the height allows red–black trees to be efficient in the worst case, unlike ordinary binary search trees.

Balancing during insertion

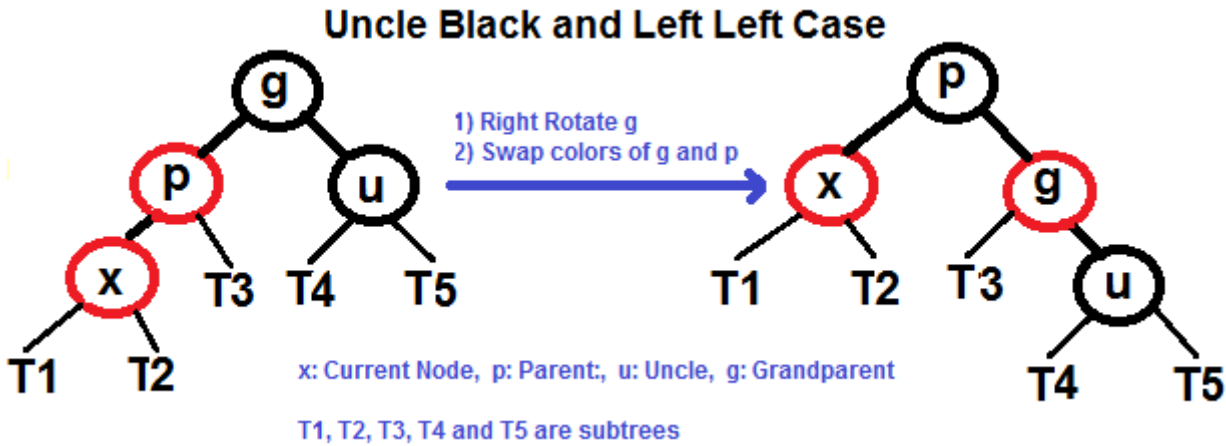
If uncle is RED



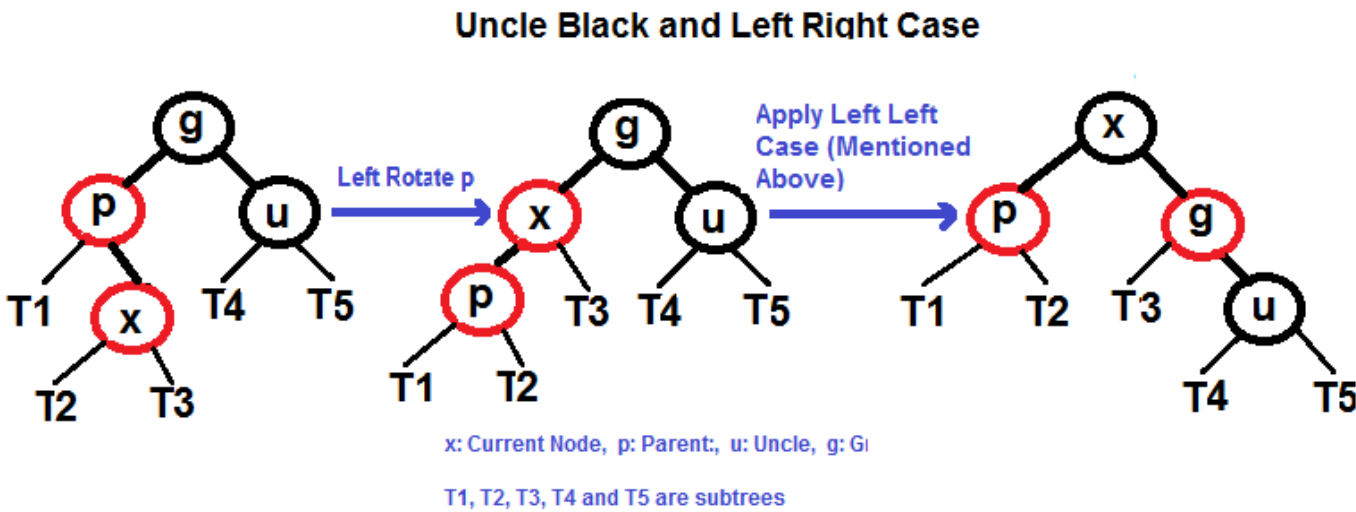
If uncle is BLACK

- Left Left Case (p is left child of g and x is left child of p)
- Left Right Case (p is left child of g and x is right child of p)
- Right Right Case (p is right child of g and x is right child of p)
- Right Left Case (p is right child of g and x is left child of p)

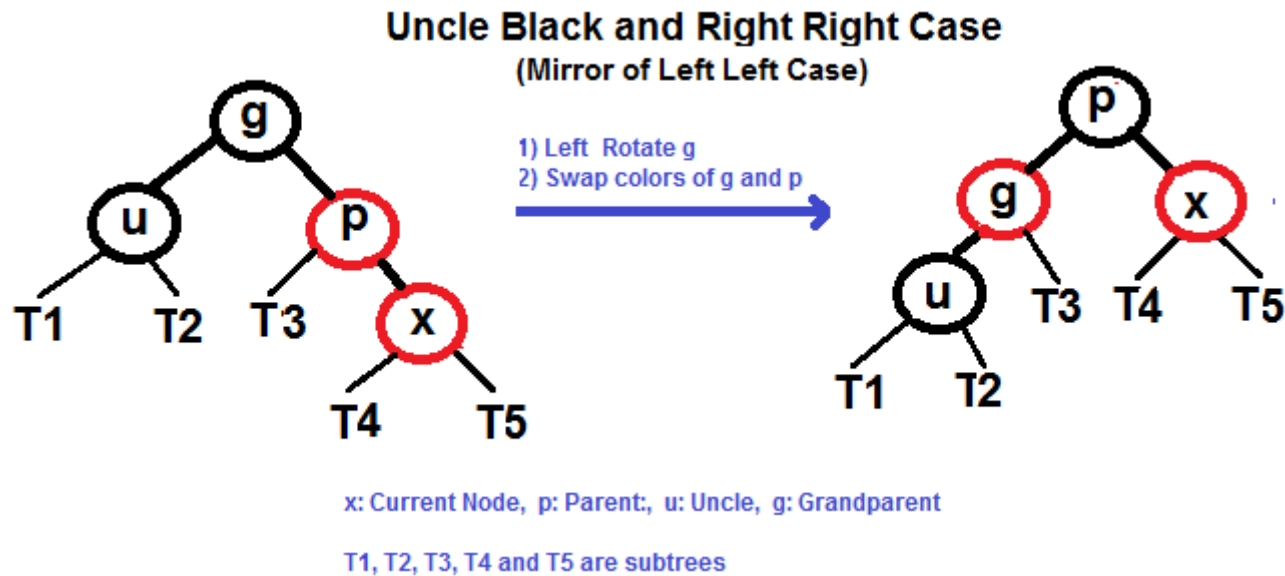
Left Left Case (See g, p and x)



Left Right Case (See g, p and x)

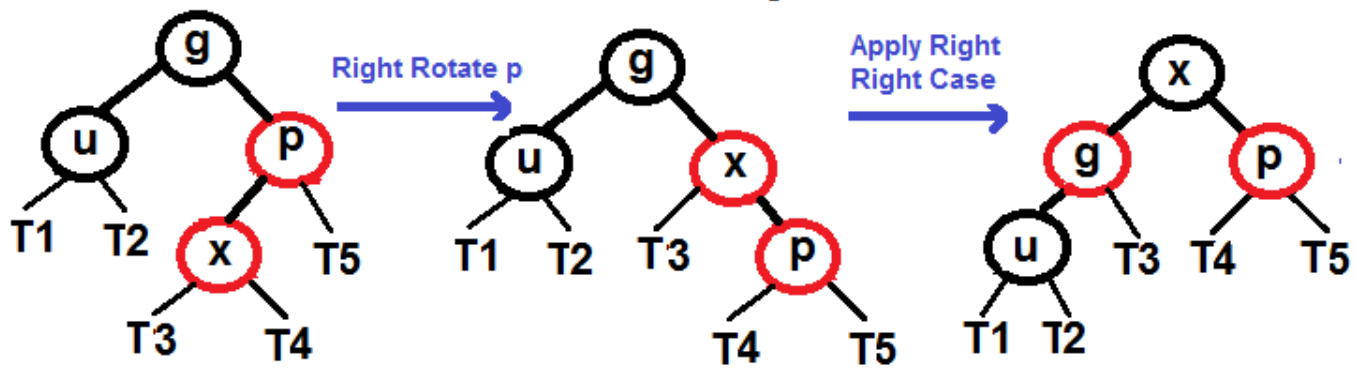


Right Right Case (See g, p and x)



Right Left Case (See g, p and x)

Uncle Black and Right Left Case (Mirror of Left Right Case)



x: Current Node, p: Parent, u: Uncle, g: Grandparent

T1, T2, T3, T4 and T5 are subtrees

Fenwick Tree / Binary Indexed Tree

A **Fenwick tree** or **binary indexed tree** is a data structure that can efficiently update elements and calculate prefix sums in a table of numbers.

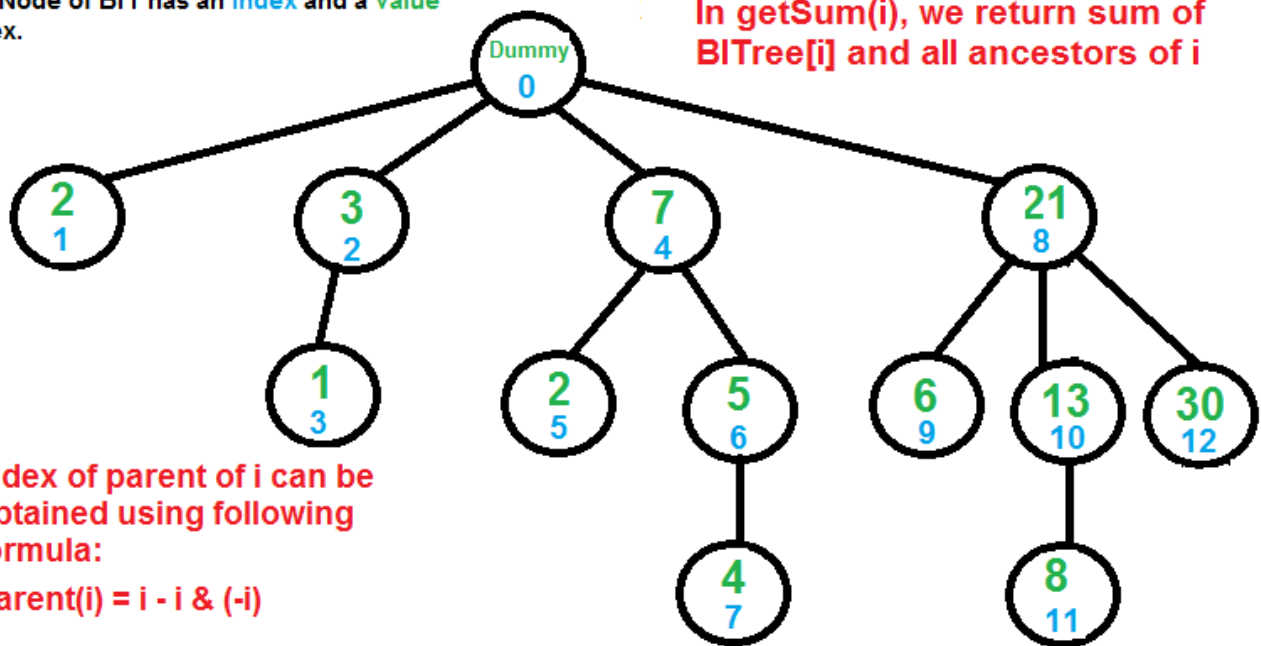
When compared with a flat array of numbers, the Fenwick tree achieves a much better balance between two operations: element update and prefix sum calculation. In a flat array of n numbers, you can either store the elements, or the prefix sums. In the first case, computing prefix sums requires linear time; in the second case, updating the array elements requires linear time (in both cases, the other operation can be performed in constant time). Fenwick trees allow both operations to be performed in $O(\log n)$ time. This is achieved by representing the numbers as a tree, where the value of each node is the sum of the numbers in that subtree. The tree structure allows operations to be performed using only $O(\log n)$ node accesses.

Implementation Notes

Binary Indexed Tree is represented as an array. Each node of Binary Indexed Tree stores sum of some elements of given array. Size of Binary Indexed Tree is equal to n where n is size of input array. In current implementation we have used size as $n+1$ for ease of implementation. All the indexes are 1-based.

Every Node of BIT has an **Index** and a **Value** at index.

In `getSum(i)`, we return sum of `BITree[i]` and all ancestors of `i`



Index of parent of `i` can be obtained using following formula:

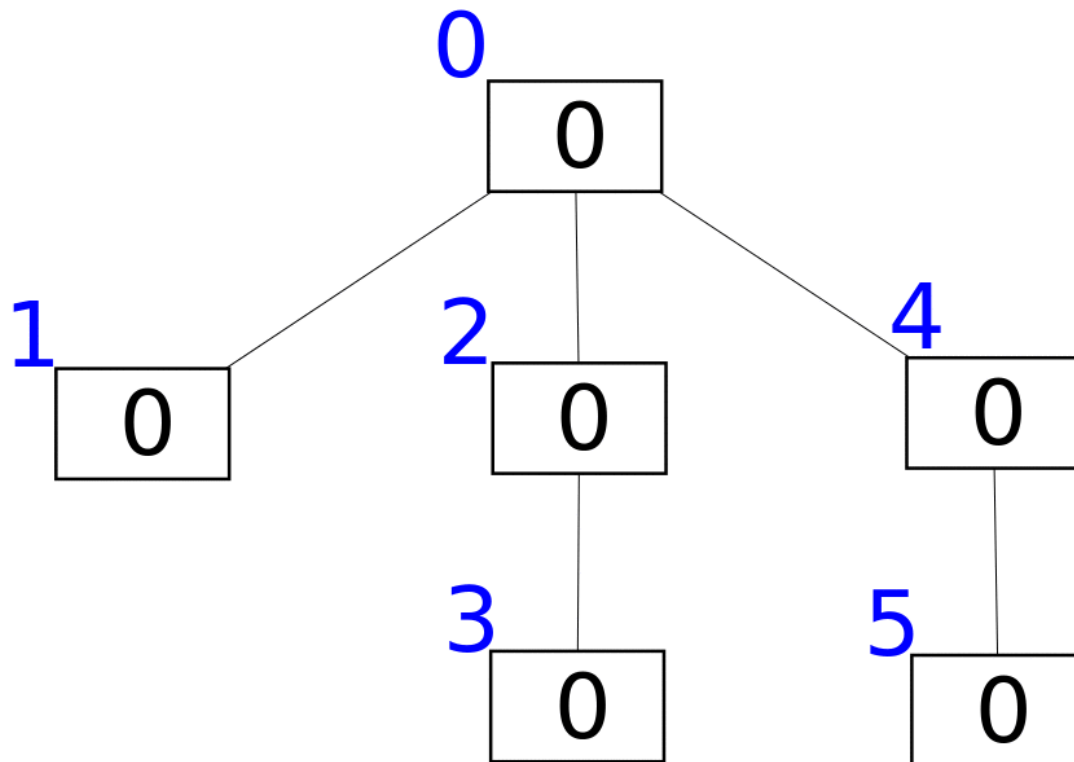
$$\text{parent}(i) = i - i \& (-i)$$

The above formula basically removes the last set bit from `i`. For example, if `i = 12`, then `parent(i)` is 8

		0	1	2	3	4	5	6	7	8	9	10	11
Input Array:	<code>arr[0..n-1]</code>	=	{2, 1, 1, 3, 2, 3, 4, 5, 6, 7, 8, 9}										
BI Tree Array:	<code>BITree[1..n]</code>	=	{2, 3, 1, 7, 2, 5, 4, 21, 6, 13, 8, 30}										
			1 2 3 4 5 6 7 8 9 10 11 12										

View of Binary Indexed Tree to understand `getSum()` operation

On the picture below you may see animated example of creation of binary indexed tree for the array `[1, 2, 3, 4, 5]` by inserting one by one.



Binary Search Tree

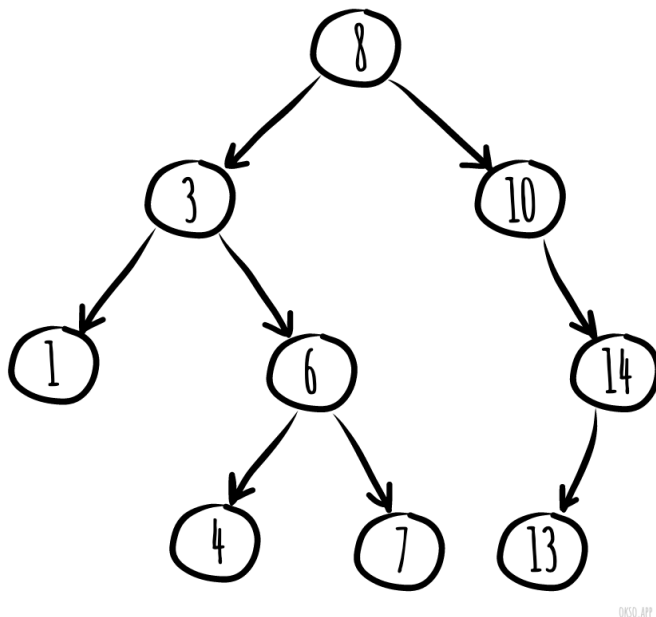
In computer science, **binary search trees** (BST), sometimes called ordered or sorted binary trees, are a particular type of container: data structures that store "items" (such as numbers, names etc.) in memory. They allow fast lookup, addition and removal of items, and can be used to implement either dynamic sets of items, or lookup tables that allow finding an item by its key (e.g., finding the phone number of a person by name).

Binary search trees keep their keys in sorted order, so that lookup and other operations can use the principle of binary search: when looking for a key in a tree (or a place to insert a new key), they traverse the tree from root to leaf, making comparisons to keys stored in the nodes of the tree and deciding, on the basis of the comparison, to continue searching in the left or right subtrees. On average, this means that each comparison allows the operations to skip about half of the tree, so that each lookup, insertion or deletion takes time proportional to the logarithm of the number of items stored in the tree. This is much better than the linear time required to find items by key in an (unsorted) array, but slower than the corresponding operations on hash tables.

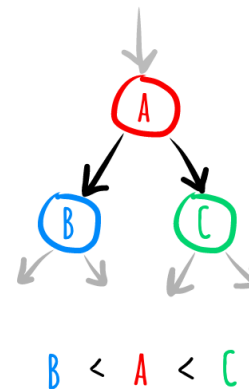
A binary search tree of size 9 and depth 3, with 8 at the root. The leaves are not drawn.

BINARY SEARCH TREE

BINARY SEARCH TREES KEEP THEIR KEYS IN SORTED ORDER, SO THAT LOOKUP AND OTHER OPERATIONS CAN USE THE PRINCIPLE OF BINARY SEARCH



FOR ANY NODE IN A BINARY SEARCH TREE:



Pseudocode for Basic Operations

Insertion

```

insert(value)
  Pre: value has passed custom type checks for type T
  Post: value has been placed in the correct location in the tree
  if root = ∅
    root ← node(value)
  else
    insertNode(root, value)
  end if
end insert
  
```

```

insertNode(current, value)
  Pre: current is the node to start from
  Post: value has been placed in the correct location in the tree
  if value < current.value
    if current.left = ∅
      current.left ← node(value)
    else
      InsertNode(current.left, value)
    end if
  else
    if current.right = ∅
      current.right ← node(value)
    end if
  end if
end insertNode
  
```

```

    else
        InsertNode(current.right, value)
    end if
end if
end insertNode

```

Searching

```

contains(root, value)
  Pre: root is the root node of the tree, value is what we would like to locate
  Post: value is either located or not
  if root = ∅
    return false
  end if
  if root.value = value
    return true
  else if value < root.value
    return contains(root.left, value)
  else
    return contains(root.right, value)
  end if
end contains

```

Deletion

```

remove(value)
  Pre: value is the value of the node to remove, root is the node of the BST
  count is the number of items in the BST
  Post: node with value is removed if found in which case yields true,
otherwise false
  nodeToRemove ← findNode(value)
  if nodeToRemove = ∅
    return false
  end if
  parent ← findParent(value)
  if count = 1
    root ← ∅
  else if nodeToRemove.left = ∅ and nodeToRemove.right = ∅
    if nodeToRemove.value < parent.value
      parent.left ← nodeToRemove.right
    else
      parent.right ← nodeToRemove.right
    end if
  else if nodeToRemove.left != ∅ and nodeToRemove.right != ∅
    next ← nodeToRemove.right
    while next.left != ∅
      next ← next.left
    end while
    if next != nodeToRemove.right

```

```

        remove(next.value)
        nodeToRemove.value ← next.value
    else
        nodeToRemove.value ← next.value
        nodeToRemove.right ← nodeToRemove.right.right
    end if
end if
else
    if nodeToRemove.left = ∅
        next ← nodeToRemove.right
    else
        next ← nodeToRemove.left
    end if
    if root = nodeToRemove
        root = next
    else if parent.left = nodeToRemove
        parent.left = next
    else if parent.right = nodeToRemove
        parent.right = next
    end if
end if
count ← count - 1
return true
end remove

```

Find Parent of Node

```

findParent(value, root)
    Pre: value is the value of the node we want to find the parent of
         root is the root node of the BST and is != ∅
    Post: a reference to the prent node of value if found; otherwise ∅
    if value = root.value
        return ∅
    end if
    if value < root.value
        if root.left = ∅
            return ∅
        else if root.left.value = value
            return root
        else
            return findParent(value, root.left)
        end if
    else
        if root.right = ∅
            return ∅
        else if root.right.value = value
            return root
        else
            return findParent(value, root.right)
        end if
    end if
end findParent

```

Find Node

```
findNode(root, value)
  Pre: value is the value of the node we want to find the parent of
       root is the root node of the BST
  Post: a reference to the node of value if found; otherwise ∅
  if root = ∅
    return ∅
  end if
  if root.value = value
    return root
  else if value < root.value
    return findNode(root.left, value)
  else
    return findNode(root.right, value)
  end if
end findNode
```

Find Minimum

```
findMin(root)
  Pre: root is the root node of the BST
       root = ∅
  Post: the smallest value in the BST is located
  if root.left = ∅
    return root.value
  end if
  findMin(root.left)
end findMin
```

Find Maximum

```
findMax(root)
  Pre: root is the root node of the BST
       root = ∅
  Post: the largest value in the BST is located
  if root.right = ∅
    return root.value
  end if
  findMax(root.right)
end findMax
```

Traversal

InOrder Traversal

```

inorder(root)
  Pre: root is the root node of the BST
  Post: the nodes in the BST have been visited in inorder
  if root != ∅
    inorder(root.left)
    yield root.value
    inorder(root.right)
  end if
end inorder

```

PreOrder Traversal

```

preorder(root)
  Pre: root is the root node of the BST
  Post: the nodes in the BST have been visited in preorder
  if root != ∅
    yield root.value
    preorder(root.left)
    preorder(root.right)
  end if
end preorder

```

PostOrder Traversal

```

postorder(root)
  Pre: root is the root node of the BST
  Post: the nodes in the BST have been visited in postorder
  if root != ∅
    postorder(root.left)
    postorder(root.right)
    yield root.value
  end if
end postorder

```

Complexities

Time Complexity

Access	Search	Insertion	Deletion
$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$

Space Complexity

$O(n)$

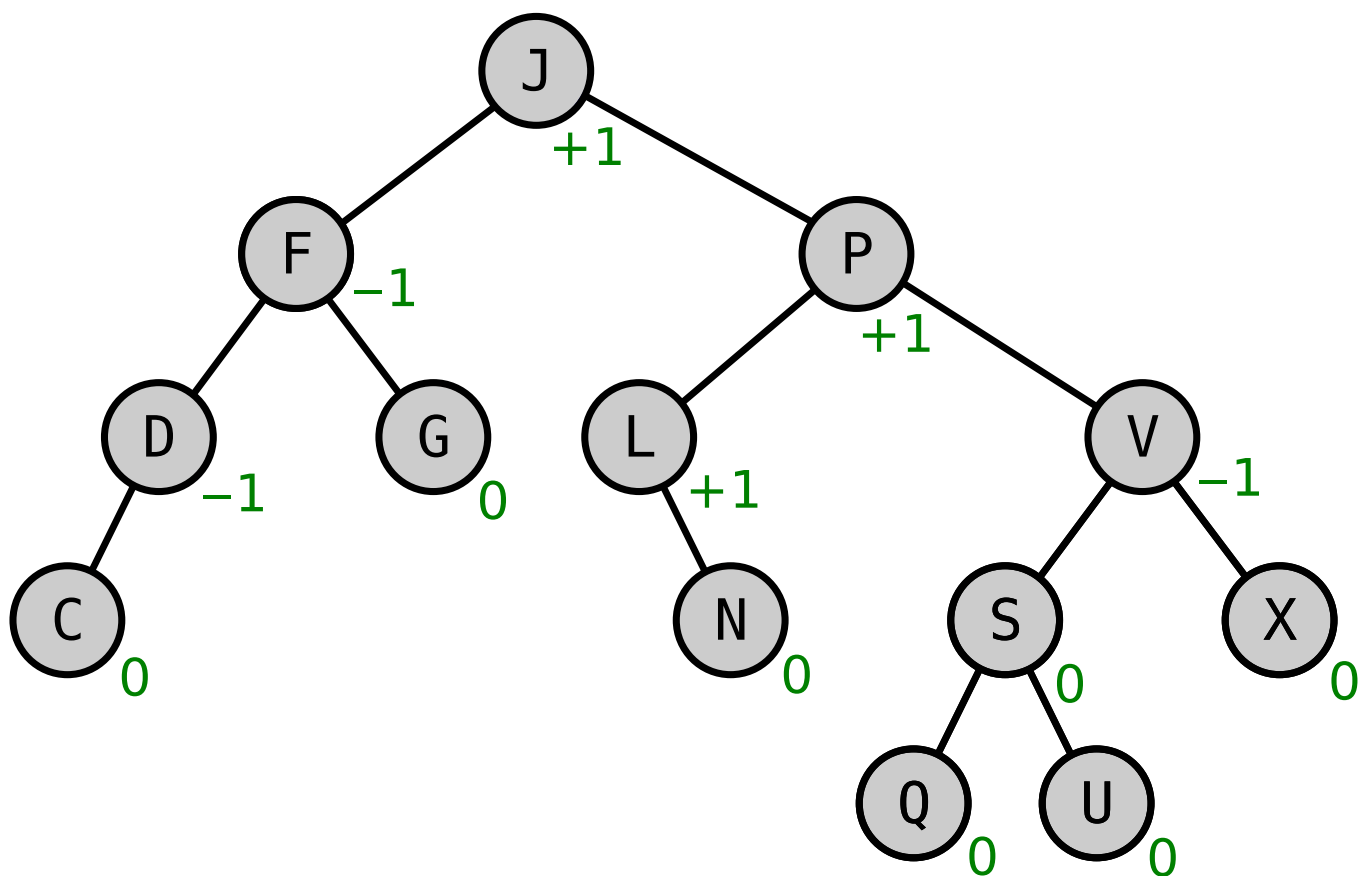
AVL Tree

In computer science, an **AVL tree** (named after inventors Adelson-Velsky and Landis) is a self-balancing binary search tree. It was the first such data structure to be invented. In an AVL tree, the heights of the two child subtrees of any node differ by at most one; if at any time they differ by more than one, rebalancing is done to restore this property. Lookup, insertion, and deletion all take $O(\log n)$ time in both the average and worst cases, where n is the number of nodes in the tree prior to the operation. Insertions and deletions may require the tree to be rebalanced by one or more tree rotations.

Animation showing the insertion of several elements into an AVL tree. It includes left, right, left-right and right-left rotations.



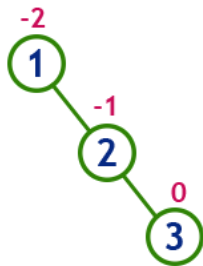
AVL tree with balance factors (green)



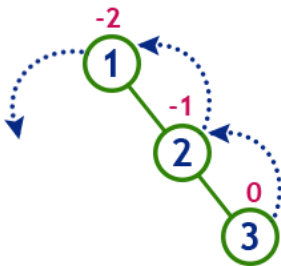
AVL Tree Rotations

Left-Left Rotation

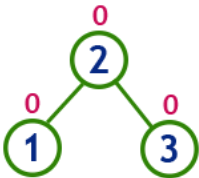
insert 1, 2 and 3



Tree is imbalanced



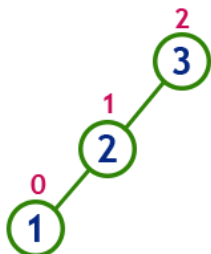
To make balanced we use LL Rotation which moves nodes one position to left



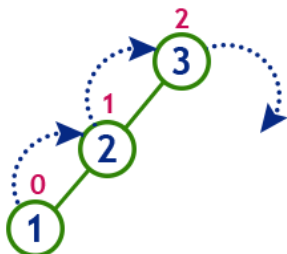
After LL Rotation
Tree is Balanced

Right-Right Rotation

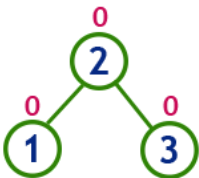
insert 3, 2 and 1



Tree is imbalanced
because node 3 has balance factor 2



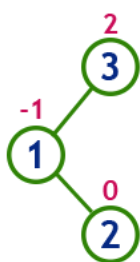
To make balanced we use RR Rotation which moves nodes one position to right



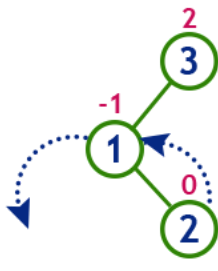
After RR Rotation
Tree is Balanced

Left-Right Rotation

insert 3, 1 and 2

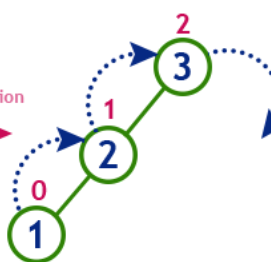


Tree is imbalanced
because node 3 has balance factor 2



LL Rotation

After LL Rotation



RR Rotation

After RR Rotation



After LR Rotation
Tree is Balanced

Right-Left Rotation

